```
'''
  What is OOP? -- Paridgm that organizes software design around obj
  Some concepts:
    * Class - Blueprint/template for creating objects,
    * Object - Instance of class (Like specific machine)
    * Attribute - Variable/Data associated with object
    * Method - Functions inside a class that perform operations
    * __init__ - Special constructir used to initialize object attr
'''


'''
  Think of a car.

  Attributes (data about the car): color, model, engine capacity
  Methods (Actions a car can do): start(), stop(), accelerate
'''


## Basic Class Example


class Machine:
  def __init__(self, name, temperature, status):
    self.name = name
    self.temperature = temperature
    self.status = status

  def check_status(self):
    if self.temperature>90:
      return f"{self.name} is overheating!"
    else:
      return f"{self.name} is operating normally."

  def display_status(self):
    print(f"{self.name} is currently {self.status}.")
```

Start coding or generate with AI.

```
## creating object - instance of class
m1 = Machine("CNC-001",95,"Running")
m2 = Machine("Drill Machine ",95,"Stopped")
```

```
m1.display_status()
m2.display_status()
```

```
CNC-001 is currently Running.
Drill Machine  is currently Stopped.
```

```
m1.temperature
```

```
95
```

```
m1.name
```

```
'CNC-001'
```

```
m1.check_status()
```

```
'CNC-001 is overheating!'
```

```
type(m1)
```

Machine
def __init__(name, temperature)

<no docstring>

```
# dir(m1)
```

```
## encapsulation - Hides internal object details.
```

```
class Sensor:
  def __init__(self):
    self.__reading = 0

  def set_reading(self,value):
    if value>=0:
      self.__reading = value

  def get_reading(self):
    return self.__reading


s = Sensor()
s.set_reading(88)
print(s.get_reading())
```

```
88
```

```
'''
Why OOP is Useful ?

* Reusability: Define once and reuse (Multiple objects from one c
* Organization: Code is structured and easy to Manage
* Scalability: Usefule for large projects (Factory system, Simulat

'''


'''
Class : Robot
Attributes: name, task
Methods: start_task(), stop_task()
'''


class Robot:
  def __init__(self, name, task):
    self.name = name
    self.task = task

  def start_task(self):
```

```
    print(f"Robot {self.name} is starting {self.task}.")

  def stop_task(self):
    print(f"Robot {self.name} has stopped {self.task}.")

## create objects
r1 = Robot("Robot1","Welding")
r2 = Robot("Robot2","Assembly")
r1.start_task()
r2.start_task()
```

```
Robot Robot1 is starting Welding.
Robot Robot2 is starting Assembly.
```

```
## self  keyword -- Represents the current object


class Product:
  def __init__(self, name, price):
    self.name = name
    self.price = price

  def apply_discount(self, percent):
    discounted_price = self.price * (1-percent/100)
    return discounted_price

p1 = Product("Gear",500)
print(f"Price after 10% discount: {p1.apply_discount(10)}")
```

```
Price after 10% discount: 450.0
```

```
### Encapsulation

'''
Hiding the internal details of object and only exposing necessary
Protect data by making variables private (_ or __) so they can't b

'''
```

```
class Machine:
  ## Use public variables -- _var, private variables -- __var
  def __init__(self, name, status, temperature):
    self.__status = status
    self.__temperature = temperature
    self.name = name

  def get_status(self):
    return self.__status

  def get_temperature(self):
    return self.__temperature

  def set_temperature(self, temp):
    if 0<= temp<=120:
      self.__temperature = temp
      print(f"Temperature of {self.name} set to {self.__temperatu
    else:
      print("Invalid temperature ! Must be between 0 and 120")


m = Machine("Lathe","Running",75)
print(m.get_temperature())
m.set_temperature(100)
m.set_temperature(200)
```

```
75
Temperature of Lathe set to 100
Invalid temperature ! Must be between 0 and 120
```

```
'''
Class: BankAccount
Private attributes: __account_number_, __balance
Methods: deposit(amount),withdraw(amount),get_balance()
'''


class BankAccount:
  def __init__(self, account_number, initial_balance=0):
    self.__account_number = account_number
    self.__balance = initial_balance
```

```
  def get_account_number(self):
    return self.__account_number

  def get_balance(self):
    return self.__balance

  def deposit(self, amount):
    if amount>0:
      self.__balance += amount
      print(f"Deposited {amount}, New Balance: {self.__balance}")
    else:
      print(f"Deposit amount must be positive !")

  def withdraw(self, amount):
    if 0<amount<=self.__balance:
      self.__balance -= amount
      print(f"Withdrawn {amount}. Remaining Balance: {self.__balar
    else:
      print(f"Insufficient balance or Invalid amount")
```

```
account= BankAccount("AAgg55511",5000)
print(f"Account Number: {account.get_account_number()}")
print(f"Current Balance: {account.get_balance()}")
```

```
Account Number: AAgg55511
Current Balance: 5000
```

```
'''
account.__balance -- Can't be accessed directly as they are privat
'''
```

```
'\naccount.__balance -- Can't be accessed directly as they ar
e private.\n'
```

```
account.deposit(2000)
```

```
Deposited 2000, New Balance: 7000
```

```
account.withdraw(3000)
```

```
Withdrawn 3000. Remaining Balance: 4000
```

```
account.withdraw(5000)
```

```
Insufficient balance or Invalid amount
```

```
'''
Inheritance - Allows a class (child class or derived class) to acc
(parent class or base class).

* code reusability
* Extensibility
* code organise
'''


class ParentClass:
  pass

class ChildClass(ParentClass):
  pass


# Car is a type of vehicle..


class Vehicle:
  def __init__(self, brand, wheels):
    self.brand = brand
    self.wheels = wheels

  def display_info(self):
    print(f"Brand: {self.brand}, Wheels: {self.wheels}")


class Car(Vehicle):
  def __init__(self, brand, wheels, model):
    ## call parent constructor
    super().__init__(brand, wheels)
```

```
    self.model = model

  def display_info(self):
    super().display_info()
    print(f"Model: {self.model}")

c1 = Car("Toyota",4,"Camry")
c1.display_info()
```

```
Brand: Toyota, Wheels: 4
Model: Camry
```

```
class Machine:
  def operate(self):
    return "Generic Operation"

class CNC(Machine):
  def operate(self):
    return "Cutting Metal"


cnc  = CNC()
print(cnc.operate())
```

```
Cutting Metal
```

```
'''
  Types of Inheritance ::

  1. Single Inheritance - One child class from one parent class
  2. Multiple - One class inherits from multiple parents classes
  3. Multilevel - child inherits from parent, which itself is a ch
  4. Hierarchical - Multiple child classes inherit from one parent
  5. Hybrid - Combination of any of above

'''



## Multiple Inheritance
```

```
class Camera:
  def take_photo(self):
    print("Taking Photo.")

class MusicPlayer:
  def play_music(self):
    print("Playic Music.")

class SmartPhone(Camera, MusicPlayer):
  def call(self):
    print("Calling a contact.")

s=SmartPhone()
s.take_photo()
s.play_music()
s.call()
```

```
Taking Photo.
Playic Music.
Calling a contact.
```

MultiLevel

```
class Device:
  def __init__(self, brand):
    self.brand = brand

  def show_brand(self):
    print(f"Brand: {self.brand}")

class Mobile(Device):
  def __init__(self, brand, sim_type):
    super().__init__(brand)
    self.sim_type = sim_type

  def show_mobile(self):
    print(f"SIM TYPE: {self.sim_type}")

class SmartPhone(Mobile):
```

```
  def __init__(self, brand, sim_type, os):
    super().__init__(brand, sim_type)
    self.os = os

  def show_smartphone(self):
    self.show_brand()
    self.show_mobile()
    print(f"Operating System: {self.os}")



s = SmartPhone("Samsung","Dual SIM","Android")
s.show_smartphone()
```

```
Brand: Samsung
SIM TYPE: Dual SIM
Operating System: Android
```

```
'''
PolyMorphism -- If a child class defines a method with the same na
'''


class Animal:
  def speak(self):
    print("Animal makes a sound.")

class Dog(Animal):
  def speak(self):
    print("Dog barks.")


a = Animal()
a.speak()
```

```
Animal makes a sound.
```

```
d = Dog()
d.speak()
```

```
Dog barks.
```